

Table des matières

Labo 04 — La base de l'ERP par ODBC, et l'API météo page par page	1
Avant de commencer — recevoir la plateforme et monter la base de l'ERP	2
Étape 1 — Interroger la base de l'ERP : filigrane, plan de requête, documents de modification	3
Étape 2 — Tirer les neuf tables de l'ERP par ODBC, puis prouver qu'elles égalent la RAW	5
Étape 3 — Vérifier le filigrane de décembre contre l'extrait.....	10
Étape 4 — Appeler l'API d'ECCC page par page, puis la comparer à l'instantané	11
Étape 5 — Versionner le code, laisser dehors données et secrets.....	15
Récapitulatif	15
Dépannage	16
Annexe — Tout le code, d'un seul tenant.....	18

Labo 04 — La base de l'ERP par ODBC, et l'API météo page par page

Labo · Module 4/12 — Formation Data Engineering

Ce que vous allez faire. Recevoir la plateforme de Boréal et monter la base de son ERP, puis l'interroger et la lire par ODBC, et prouver que les extraits mensuels chargés au Cours 03 racontent la même histoire qu'elle. Tirer 92 jours de météo de la vraie API d'Environnement et Changement climatique Canada, et les comparer à l'instantané. Objectif de fin : 0 écart entre la base et les extraits sur les neuf tables de l'ERP, le filigrane de décembre égal à l'extrait, et 92 jours reçus sur 92 annoncés.

Trois terminaux servent dans ce labo, et chaque étape dit lequel. L'Anaconda Prompt, environnement `formations` actif, sert à Python et à Git. La ligne de commande DuckDB sert au SQL de vérification. Ubuntu, dans WSL 2, sert à Docker et à tout ce qui touche la base de l'ERP dans son conteneur : c'est là que vit Docker Engine, et nulle part ailleurs.

La liste de contrôle du cours :

- ☐ Fichiers de la plateforme copiés dans `Projet_Boreal/plateforme/`, base de l'ERP démarrée et remplie : 25 483 lignes dans VBAK
- ☐ La base de l'ERP interrogée sur le port 5433 : 25 483 commandes, un filigrane de 427
- ☐ `01_de/.env` écrit ; `extraire_erp_odbc.py` lancé ; `controle_erp_base.sql` : 0 écart sur les neuf tables
- ☐ Le filigrane de décembre 2025 égal à l'extrait de décembre, ligne pour ligne

- ☐ extraire_meteo_api.py lancé : 92 jours en 2 pages ; 0 écart avec l'instantané, comparé en nombres
 - ☐ Les scripts, le contrôle et .env.exemple versionnés ; .env hors du dépôt
-

Avant de commencer — recevoir la plateforme et monter la base de l'ERP

Ce cours lit la base de l'ERP de Boréal, qui tourne dans un conteneur. C'est ici qu'on la reçoit et qu'on la monte, une seule fois : le reste du labo ne fait plus que l'interroger. Il faut le socle du parcours, Docker dans l'Ubuntu de WSL 2 et le pilote psqLODBC compris ; Data_Domain/GUIDE_INSTALLATION.md le monte, étapes 5 et 7. La RAW du Cours 03, enfin, doit être chargée : l'étape 2 la compare à la base.

La plateforme de Boréal ne s'écrit pas : elle se reçoit. Ses fichiers vivent dans _univers/plateforme/, à côté du générateur : le docker-compose.yml qui décrit ses services (la base de l'ERP et Airflow), le Dockerfile.airflow qui construit l'image d'Airflow, et le modèle de son .env. En entreprise, c'est l'équipe qui exploite la plateforme qui les écrit et les tient : vous les copiez sans les modifier. Ce cours explique le service de l'ERP, le Cours 06 ceux d'Airflow.

Dans l'Anaconda Prompt, depuis Projet_Boreal, où le dossier plateforme existe depuis le Cours 02 :

```
copy ..\_univers\plateforme\docker-compose.yml plateforme\  
copy ..\_univers\plateforme\Dockerfile.airflow plateforme\  
copy ..\_univers\plateforme\.env.exemple plateforme\.env  
notepad plateforme\.env
```

Le Bloc-notes ouvre le .env de la plateforme. Remplacez remplacez-moi par un mot de passe de votre choix pour la base de l'ERP, puis enregistrez et fermez :

ERP_MOT_DE_PASSE=choisissez-un-mot-de-passe

Piège — un mot de passe qui casse la lecture du .env. Tenez-vous aux lettres et aux chiffres. Un # ferait passer la suite de la ligne pour un commentaire, un \$ serait pris pour une variable à remplacer, et des guillemets entreraient dans la valeur. Notez votre mot de passe quelque part : la base est initialisée avec lui au premier démarrage, et l'étape 2 le redemande.

dir /a plateforme doit montrer trois fichiers neufs à côté du lac : .env, docker-compose.yml et Dockerfile.airflow. Aucune commande ne nomme le .env : docker compose lit tout seul celui qui se trouve à côté du docker-compose.yml, et c'est pour cette raison qu'ils vivent dans le même dossier.

Passez dans Ubuntu, dans le dossier de la plateforme, et démarrez la base de l'ERP, seule :

```
cd plateforme  
wsl  
  
docker compose up -d erp  
docker compose ps
```

Au premier lancement, Docker télécharge l'image de PostgreSQL 18, puis crée un réseau, le volume `boreal_erp_donnees` et le conteneur `boreal_erp`. `docker compose ps` le montre Up, avec `0.0.0.0:5433->5432/tcp` : la base répond sur le port 5433 de la machine, et le 5432 reste libre pour un PostgreSQL déjà installé. PostgreSQL initialise sa base pendant quelques secondes.

Piège — le mot de passe n'est lu qu'une fois. PostgreSQL applique `POSTGRES_PASSWORD` quand il initialise sa base, c'est-à-dire au tout premier démarrage du volume, et jamais ensuite. Si vous modifiez le `.env` après coup, le serveur garde son ancien mot de passe : tout marche encore par `docker compose exec`, qui entre par le conteneur sans en fournir, mais la lecture par ODBC de l'étape 2 échoue sur `password authentication failed for user "postgres"`. Deux sorties, dans Ubuntu depuis plateforme. Aligner le serveur sur le `.env`, sans rien perdre : `docker compose exec erp psql -U postgres -d erp_boreal -c "ALTER USER postgres PASSWORD 'votre-valeur' ;"`. Ou repartir à neuf : `docker compose down -v`, `docker compose up -d erp`, ce qui efface la base et oblige à la recharger.

Chargez maintenant la copie de la base livrée par l'ERP, avec le script que l'ERP a livré :

```
docker compose exec -w /erp erp psql -U postgres -d erp_boreal -f charger_erp.sql
```

`docker compose exec` lance une commande dans le conteneur du service `erp` ; `-w /erp` la lance depuis le dossier monté, où `charger_erp.sql` trouve ses CSV. Le script prévient d'abord que le schéma `sap` n'existe pas encore (NOTICE: schema "sap" does not exist, skipping), puis affiche les comptes :

table_sap	lignes
CDHDR	53
CDPOS	53
KNA1	156
MAKT	136
MARA	68
T005U	20
VBAK	25483
VBAP	77732
ZSERIE	27775

(9 rows)

Au second passage, le message d'ouverture change : le schéma existe, et le script annonce qu'il le supprime (NOTICE: drop cascades to 9 other objects) avant de le recréer. C'est voulu : ce script se rejoue autant de fois qu'on veut, et rend toujours les mêmes comptes.

La base tourne. Laissez la fenêtre Ubuntu ouverte, et revenez à l'Anaconda Prompt, dans `Projet_Boreal`, pour l'étape 1.

Étape 1 — Interroger la base de l'ERP : filigrane, plan de requête, documents de modification

Théorie : `Theorie_04.md`, sections 2 et 3.

La base de l'ERP tourne déjà : le bloc d'ouverture de ce labo l'a démarrée dans son conteneur et remplie avec le script que l'ERP a livré. PostgreSQL 18 répond sur le port 5433 de la machine, et le schéma `sap` porte les neuf tables du dictionnaire. Ici, on l'interroge.

Si vous avez fermé la fenêtre Ubuntu, revenez-y, dans le dossier de la plateforme. Depuis `Projet_Boreal`, dans l'Anaconda Prompt :

```
cd plateforme
wsl
```

Ubuntu s'ouvre dans le même dossier, que l'invite affiche en `/mnt/c/.../plateforme`. C'est de là que `docker compose` travaille : le `docker-compose.yml` et le `.env` qui porte le mot de passe y vivent ensemble, et aucune commande n'a besoin de les nommer.

Piège — Ubuntu qui s'arrête dans votre dos. WSL arrête Ubuntu peu après la fermeture du dernier terminal Ubuntu, et les conteneurs avec elle. Le délai se règle une fois pour toutes à l'étape 6 du guide d'installation du parcours ; à défaut, gardez cette fenêtre ouverte jusqu'à la fin de l'étape 2. Si vous l'avez fermée, rouvrez Ubuntu dans `plateforme` : `restart` : `unless-stopped` fait repartir le conteneur, avec ses données.

Vérifiez que la base répond :

```
docker compose ps erp
```

Le conteneur `boreal_erp` doit être Up, avec `0.0.0.0:5433->5432/tcp`. S'il ne l'est pas, `docker compose up -d erp` le redémarre ; s'il n'existe pas, reprenez le bloc d'ouverture de ce labo.

Ouvrez `psql`, le client de PostgreSQL, dans le conteneur :

```
docker compose exec erp psql -U postgres -d erp_boreal
```

L'invite devient `erp_boreal=#`. Le SQL qui suit se tape là, dans PostgreSQL : ni dans DuckDB, ni dans Ubuntu. Collez chaque requête d'un bloc, commentaire compris, et jusqu'au point-virgule : dans `psql`, `--` ouvre un commentaire SQL, et rien ne part avant le `;`. Pour ressortir, `\q`.

Piège — du SQL collé dans le terminal d'Ubuntu. Trois invites se ressemblent, et chacune parle une langue : `erp_boreal=#`, c'est PostgreSQL ; `D`, c'est DuckDB ; une ligne qui finit par `$`, par exemple `utilisateur@poste:.../plateforme$`, c'est Ubuntu, qui ne connaît pas le SQL. Collé là, un `SELECT` répond `SELECT: command not found`, ou `syntax error near unexpected token`. Pire, le shell prend le `>=` du `WHERE` pour une redirection et crée un fichier vide nommé `=` dans `plateforme` : supprimez-le avec `rm ./=`. Regardez l'invite avant de coller.

Le filigrane de décembre 2025, d'abord : les commandes créées ou modifiées depuis le 1er décembre.

```
-- Attendu : 427.
SELECT COUNT(*) AS decembre
FROM sap.vbak
WHERE erdat >= '20251201' OR aedat >= '20251201';
```

Puis ce que coûte ce filigrane à la base. `EXPLAIN` montre le plan que PostgreSQL choisit, sans exécuter la requête :

```
-- Attendu : Index Scan using ix_vbak_aedat on vbak.
EXPLAIN SELECT * FROM sap.vbak WHERE aedat >= '20251201';
```

```
-- Attendu : Seq Scan on vbak.  
EXPLAIN SELECT * FROM sap.vbak WHERE erdat >= '20251201' OR aedat >= '20251201';
```

La base porte un index sur aedat : un filtre sur cette seule date passe par lui. Le filtre complet, avec son OR sur erdat, qui n'a pas d'index, lit toute la table. Sur 25 483 commandes, c'est instantané ; sur une vraie base de production, c'est la requête qu'on n'envoie pas à midi.

Les documents de modification, enfin : qui a changé quoi, et quand.

```
-- Attendu : 5 lignes. 20221117 XD02 LOEVM 1, 20230717 XD02 LOEVM 1, 20240701 ZPALIER KUKLA 49,  
-- 20241213 XD02 LOEVM 1, 20250410 XD02 LOEVM 1.  
SELECT h.ude, h.tcode, p.fname, COUNT(*) AS changements  
FROM sap.cdhdr h  
JOIN sap.cdpos p USING (mandt, objectclas, objectid, changenr)  
GROUP BY h.ude, h.tcode, p.fname  
ORDER BY h.ude;
```

Les 49 changements de palier du 2024-07-01 viennent du programme de révision ZPALIER ; les quatre fermetures de concessionnaires ont été saisies à la main par XD02. Quittez psql avec \q, et laissez la fenêtre Ubuntu ouverte.

Étape 2 — Tirer les neuf tables de l'ERP par ODBC, puis prouver qu'elles égalent la RAW

Théorie : Theorie_04.md, sections 1 et 2.

Le script d'extraction tourne sous Windows et parle à la base par ODBC. Le pilote psqLODBC et le paquet pyodbc sont installés depuis le guide du parcours (étapes 2 et 7) ; ce qui manque est le mot de passe, que le code doit lire quelque part sans l'écrire dans le dépôt.

Votre dépôt reçoit donc son propre .env, distinct de celui de la plateforme. Celui de la plateforme sert à docker compose, qui initialise la base ; celui-ci sert à votre code, qui s'y connecte. Ouvrez une nouvelle fenêtre de l'Anaconda Prompt, en laissant la fenêtre Ubuntu ouverte, activez formations, placez-vous dans Projet_Boreal, puis :

```
notepad 01_de\.env
```

Le fichier n'existe pas encore : Notepad propose de le créer, répondez oui. Écrivez-y une seule ligne, avec le mot de passe choisi à l'installation, puis enregistrez et fermez :

```
ERP_MOT_DE_PASSE=celui-du-.env-de-la-plateforme
```

Puis son modèle, sans secret, qui ira dans le dépôt pour dire aux autres quoi remplir :

```
notepad 01_de\.env.exemple
```

```
# Modèle du .env : copiez-le en .env et remplacez la valeur par le mot de passe de la base de l'ERP,  
# celui du .env de la plateforme. Ce modèle va dans Git, le .env jamais.  
ERP_MOT_DE_PASSE=remplacez-moi
```

Pourquoi — deux fichiers pour un seul mot de passe. Le `.env` ne quitte jamais votre poste : personne d'autre ne peut donc savoir de quelles variables le projet a besoin. Le `.env.exemple`, lui, est versionné, et porte la liste de ces noms avec des valeurs bidon. Qui clone le dépôt le copie en `.env`, met ses propres valeurs, et démarre. La convention se retrouve partout, de dbt à Airflow : un dépôt sans son `.env.exemple` est un dépôt qu'on ne sait pas lancer.

Piège — le mot de passe qui ne concorde pas. Si la base a été initialisée avec une autre valeur que celle que vous venez d'écrire, le script échouera sur `password authentication failed for user "postgres"`. Le dépannage du guide d'installation donne les deux sorties.

Vérifiez d'abord quels pilotes ODBC le poste connaît :

```
python -c "import pyodbc; print(pyodbc.drivers())"
```

La liste des pilotes ODBC du poste doit contenir un pilote dont le nom commence par PostgreSQL Unicode. Sur le poste de l'auteur, quatre entrées PostgreSQL apparaissent, deux jeux de caractères pour deux architectures :

```
'PostgreSQL ANSI(x64)', 'PostgreSQL ANSI', 'PostgreSQL Unicode(x64)', 'PostgreSQL Unicode'
```

Le script qui suit garde la première qui commence par PostgreSQL Unicode, ici PostgreSQL Unicode(x64) : Unicode parce que les libellés de l'ERP portent des accents, et 64 bits parce que le Python de formations l'est. Si aucune n'apparaît, reprenez l'étape 7 du guide d'installation : le pilote manque, ou c'est la version 32 bits qui a été prise. Créez le script :

```
notepad 01_de\ingestion\extraire_erp_odbc.py
```

```
# -*- coding: utf-8 -*-
```

```
"""extraire_erp_odbc.py : Les neuf tables de la base de l'ERP, lues par ODBC, déposées dans Le bronze.
```

```
Lancement, depuis Projet_Boreal/01_de, Le conteneur démarré : python ingestion/extraire_erp_odbc.py
"""
```

```
import csv
import sys
import time
from pathlib import Path
```

```
import duckdb
import pyodbc
```

```
PROJET = Path(__file__).resolve().parents[2]
TABLES = ["t005u", "kna1", "mara", "makt", "vbak", "vbap", "cdhdr", "cdpos", "zserie"]
PAQUET = 10_000
```

```
def lire_env(cle: str) -> str:
    """Une valeur de 01_de/.env, un fichier de lignes CLE=valeur que Git ignore."""
    env = PROJET / "01_de" / ".env"
```

```

if env.exists():
    for ligne in env.read_text(encoding="utf-8").splitlines():
        nom, _, valeur = ligne.partition("=")
        if nom.strip() == cle and not ligne.lstrip().startswith("#"):
            return valeur.strip()
sys.exit(f"{cle} introuvable dans {env} : copiez .env.exemple en .env et remplissez-le.")

def main():
    pilotes = [p for p in pyodbc.drivers() if p.startswith("PostgreSQL Unicode")]
    if not pilotes:
        sys.exit(f"Pilote psqLODBC introuvable. Pilotes ODBC installés : {pyodbc.drivers()}")
    chaine = (f"DRIVER={{pilotes[0]}};SERVER=localhost;PORT=5433;DATABASE=erp_boreal;"
             f"UID=postgres;PWD={lire_env('ERP_MOT_DE_PASSE')}")
    sortie = PROJET / "plateforme" / "lac" / "bronze" / "erp_base"
    sortie.mkdir(parents=True, exist_ok=True)

    debut = time.perf_counter()
    connexion = pyodbc.connect(chaine)
    curseur = connexion.cursor()
    duck = duckdb.connect()
    for table in TABLES:
        curseur.execute(f"SELECT * FROM sap.{table}")
        colonnes = [c[0].upper() for c in curseur.description]
        temporaire = sortie / f"{table}.csv"
        lignes = 0
        with open(temporaire, "w", newline="", encoding="utf-8") as fichier:
            ecrivain = csv.writer(fichier)
            ecrivain.writerow(colonnes)
            while paquet := curseur.fetchmany(PAQUET):
                ecrivain.writerows(paquet)
                lignes += len(paquet)
        # Le bronze garde Le texte reçu ; DuckDB Le range en Parquet compressé.
        duck.execute(f"""
            COPY (SELECT * FROM read_csv('{temporaire.as_posix()}', header = true, all_varchar = true))
            TO '{(sortie / f"{table}.parquet").as_posix()}' (FORMAT parquet, COMPRESSION zstd)""")
        temporaire.unlink()
        print(f"sap.{table:7} {lignes:>7} lignes")
    connexion.close()
    print(f"Neuf tables extraites en {time.perf_counter() - debut:.1f} s : "
          f"{sortie.relative_to(PROJET).as_posix()}")

```

```
if __name__ == "__main__":
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")
    main()
```

Morceau	Ce qu'il fait
lire_env	lit le mot de passe dans 01_de/.env : il n'est écrit ni dans le code, ni dans le dépôt
la chaîne de connexion	le pilote, le serveur, le port 5433, la base, l'utilisateur et le mot de passe
fetchmany(PAQUET)	lit 10 000 lignes à la fois : la mémoire du script ne dépend pas de la taille de la table
le CSV temporaire, puis COPY	le texte reçu, rangé en Parquet par DuckDB ; le CSV part ensuite

Lancez-le depuis 01_de :

```
cd 01_de
python ingestion\extraire_erp_odbc.py
cd ..
```

Une ligne par table, de sap.t005u (20 lignes) à sap.zserie (27 775 lignes), puis la durée totale. Neuf fichiers Parquet attendent dans plateforme/lac/bronze/erp_base/. Reste la preuve : les extraits mensuels chargés dans la RAW retrouvent-ils la base ? Créez le script de contrôle :

```
notepad 01_de\sql\controle_erp_base.sql
```

```
-- 01_de/sql/controle_erp_base.sql : Les extraits de la RAW retrouvent-ils la base de L'ERP ?
-- Lancement, depuis Projet_Boreal, la ligne de commande DuckDB fermée : duckdb -f 01_de/sql/controle_erp_base.sql
```

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
USE entrepot;
```

WITH

```
-- La dernière version de chaque clé, colonnes de lignage retirées. Les clés sont celles du dictionnaire.
t005u AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_t005u
          QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, SPRAS, LAND1, BLAND ORDER BY _lot DESC) = 1),
kna1 AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_kna1
          QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, KUNNR ORDER BY _lot DESC) = 1),
mara AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_mara
          QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, MATNR ORDER BY _lot DESC) = 1),
makt AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_makt
          QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, MATNR, SPRAS ORDER BY _lot DESC) = 1),
```



```

vbak AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_vbak
        QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, VBELN ORDER BY _lot DESC) = 1),
vbap AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_vbap
        QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, VBELN, POSNR ORDER BY _lot DESC) = 1),
cdhdr AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_cdhdr
        QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, OBJECTCLAS, OBJECTID, CHANGENR ORDER BY _lot DESC) = 1),
cdpos AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_cdpos
        QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, OBJECTCLAS, OBJECTID, CHANGENR, TABNAME, TABKEY, FNAME, CHNGIND ORDER BY
        _lot DESC) = 1),
zserie AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_zserie
        QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, SERNR ORDER BY _lot DESC) = 1),
-- La même table, lue dans la base par ODBC.
b_t005u AS (FROM 'plateforme/lac/bronze/erp_base/t005u.parquet'),
b_kna1 AS (FROM 'plateforme/lac/bronze/erp_base/kna1.parquet'),
b_mara AS (FROM 'plateforme/lac/bronze/erp_base/mara.parquet'),
b_makt AS (FROM 'plateforme/lac/bronze/erp_base/makt.parquet'),
b_vbak AS (FROM 'plateforme/lac/bronze/erp_base/vbak.parquet'),
b_vbap AS (FROM 'plateforme/lac/bronze/erp_base/vbap.parquet'),
b_cdhdr AS (FROM 'plateforme/lac/bronze/erp_base/cdhdr.parquet'),
b_cdpas AS (FROM 'plateforme/lac/bronze/erp_base/cdpas.parquet'),
b_zserie AS (FROM 'plateforme/lac/bronze/erp_base/zserie.parquet')
SELECT 'T005U' AS table_sap, (SELECT COUNT(*) FROM t005u) AS lignes_raw, (SELECT COUNT(*) FROM b_t005u) AS lignes_base,
        (SELECT COUNT(*) FROM (FROM t005u EXCEPT FROM b_t005u)) AS raw_sans_base,
        (SELECT COUNT(*) FROM (FROM b_t005u EXCEPT FROM t005u)) AS base_sans_raw
UNION ALL
SELECT 'KNA1', (SELECT COUNT(*) FROM kna1), (SELECT COUNT(*) FROM b_kna1),
        (SELECT COUNT(*) FROM (FROM kna1 EXCEPT FROM b_kna1)), (SELECT COUNT(*) FROM (FROM b_kna1 EXCEPT FROM kna1))
UNION ALL
SELECT 'MARA', (SELECT COUNT(*) FROM mara), (SELECT COUNT(*) FROM b_mara),
        (SELECT COUNT(*) FROM (FROM mara EXCEPT FROM b_mara)), (SELECT COUNT(*) FROM (FROM b_mara EXCEPT FROM mara))
UNION ALL
SELECT 'MAKT', (SELECT COUNT(*) FROM makt), (SELECT COUNT(*) FROM b_makt),
        (SELECT COUNT(*) FROM (FROM makt EXCEPT FROM b_makt)), (SELECT COUNT(*) FROM (FROM b_makt EXCEPT FROM makt))
UNION ALL
SELECT 'VBAK', (SELECT COUNT(*) FROM vbak), (SELECT COUNT(*) FROM b_vbak),
        (SELECT COUNT(*) FROM (FROM vbak EXCEPT FROM b_vbak)), (SELECT COUNT(*) FROM (FROM b_vbak EXCEPT FROM vbak))
UNION ALL
SELECT 'VBAP', (SELECT COUNT(*) FROM vbap), (SELECT COUNT(*) FROM b_vbap),
        (SELECT COUNT(*) FROM (FROM vbap EXCEPT FROM b_vbap)), (SELECT COUNT(*) FROM (FROM b_vbap EXCEPT FROM vbap))
UNION ALL
SELECT 'CDHDR', (SELECT COUNT(*) FROM cdhdr), (SELECT COUNT(*) FROM b_cdhdr),
        (SELECT COUNT(*) FROM (FROM cdhdr EXCEPT FROM b_cdhdr)), (SELECT COUNT(*) FROM (FROM b_cdhdr EXCEPT FROM cdhdr))

```

```

UNION ALL
SELECT 'CDPOS', (SELECT COUNT(*) FROM cdpos), (SELECT COUNT(*) FROM b_cdpos),
  (SELECT COUNT(*) FROM (FROM cdpos EXCEPT FROM b_cdpos)), (SELECT COUNT(*) FROM (FROM b_cdpos EXCEPT FROM cdpos))
UNION ALL
SELECT 'ZSERIE', (SELECT COUNT(*) FROM zserie), (SELECT COUNT(*) FROM b_zserie),
  (SELECT COUNT(*) FROM (FROM zserie EXCEPT FROM b_zserie)), (SELECT COUNT(*) FROM (FROM b_zserie EXCEPT FROM zserie));

```

La requête garde, pour chaque clé de chaque table, la version du lot le plus récent (QUALIFY ROW_NUMBER() ... = 1) : c'est la règle de la Théorie 03, section 2, qui retrouve l'état d'une base dans des extraits incrémentaux. Puis elle compare cet état à la base, dans les deux sens (EXCEPT). FROM 'fichier.parquet' est la forme courte de SELECT * FROM. Lancez le contrôle, depuis Projet_Boreal :

```
duckdb -f 01_de/sql/controle_erp_base.sql
```

table_sap varchar	lignes_raw int64	lignes_base int64	raw_sans_base int64	base_sans_raw int64
T005U	20	20	0	0
KNA1	156	156	0	0
MARA	68	68	0	0
MAKT	136	136	0	0
VBAK	25483	25483	0	0
VBAP	77732	77732	0	0
CDHDR	53	53	0	0
CDPOS	53	53	0	0
ZSERIE	27775	27775	0	0

Neuf tables, 0 écart dans les deux sens. Deux voies d'extraction indépendantes, les extraits mensuels et la base lue par ODBC, racontent la même histoire. Les champs vides concordent aussi : la base stocke des chaînes vides, le CSV les transporte en champs vides, et DuckDB les lit en NULL des deux côtés.

À retenir — la reconstruction est une preuve, pas une livraison. Garder la version la plus récente de chaque clé est une transformation : la RAW ne la fait pas, elle garde toutes les versions. Ce contrôle la fait seulement pour prouver que rien ne manque. Le STAGING d'Analytics Engineering la refera, pour de bon.

Vous pouvez maintenant fermer la fenêtre Ubuntu : la suite du labo n'a plus besoin de la base.

Étape 3 — Vérifier le filigrane de décembre contre l'extrait

Théorie : Theorie_04.md, section 3.

À l'étape 1, la base a rendu 427 commandes créées ou modifiées depuis le 1er décembre 2025. L'extrait mensuel de décembre en compte-t-il autant, et les mêmes ? Rouvrez DuckDB (duckdb) :

```
-- Attendu : 427.
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
USE entrepot;
SELECT COUNT(*) AS en_tetes FROM raw.erp_vbak WHERE _lot = '2025-12';
```

L'état de la base au 2025-12-31, que l'étape 2 a prouvé égal à la RAW reconstruite, se découpe ainsi :

```
-- Attendu : 427, 37, 390 et 9.
WITH etat AS (
    SELECT * FROM raw.erp_vbak
    QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, VBELN ORDER BY _lot DESC) = 1)
SELECT COUNT(*) FILTER (WHERE ERDAT >= '20251201' OR AEDAT >= '20251201') AS filigrane,
    COUNT(*) FILTER (WHERE AEDAT >= '20251201') AS modifiees_en_decembre,
    COUNT(*) FILTER (WHERE ERDAT >= '20251201' AND AEDAT = '00000000') AS creees_jamais_modifiees,
    COUNT(*) FILTER (WHERE ERDAT < '20251201' AND AEDAT >= '20251201') AS anciennes_modifiees
FROM etat;
```

Un filigrane posé sur la seule date de modification rendrait 37 commandes, et raterait les 390 commandes créées en décembre, jamais modifiées, dont AEDAT vaut 00000000. Les deux dates ensemble rendent 427. Reste à prouver que ce sont les mêmes lignes que l'extrait :

```
-- Attendu : 0 et 0.
WITH etat AS (
    SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_vbak
    QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, VBELN ORDER BY _lot DESC) = 1),
    filigrane AS (SELECT * FROM etat WHERE ERDAT >= '20251201' OR AEDAT >= '20251201'),
    extrait AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_vbak WHERE _lot = '2025-12')
SELECT (SELECT COUNT(*) FROM (FROM filigrane EXCEPT FROM extrait)) AS filigrane_sans_extrait,
    (SELECT COUNT(*) FROM (FROM extrait EXCEPT FROM filigrane)) AS extrait_sans_filigrane;
```

La capture par filigrane sur la base et l'extrait mensuel donnent exactement les mêmes 427 lignes : l'extrait de l'ERP n'est rien d'autre que cette requête, lancée la nuit qui suit la fin du mois. Quittez DuckDB (.exit).

Étape 4 — Appeler l'API d'ECCC page par page, puis la comparer à l'instantané

Théorie : Theorie_04.md, sections 1 et 4.

La météo de la RAW vient d'un instantané figé. Tirez maintenant la même donnée de la vraie API, page par page. Dans l'Anaconda Prompt, depuis Projet_Boreal, la connexion Internet active :

```
notepad 01_de\ingestion\extraire_meteo_api.py

# -*- coding: utf-8 -*-
"""extraire_meteo_api.py : La vraie API d'ECCC, page par page, déposée dans Le bronze.
```

Lancement, depuis Projet_Boreal/01_de : `python ingestion/extraire_meteo_api.py`

```
"""
import json
import sys
import time
import urllib.error
import urllib.parse
import urllib.request
from pathlib import Path

PROJET = Path(__file__).resolve().parents[2]
API = "https://api.weather.gc.ca/collections/climate-daily/items"
STATION, DEBUT, FIN = "7060400", "2025-10-01", "2025-12-31"
PAGE = 50
ESSAIS = 4

def lire_page(url: str) -> dict:
    """Une page de l'API. Réessaie après une coupure, un 429 ou une erreur 5xx."""
    for essai in range(1, ESSAIS + 1):
        try:
            with urllib.request.urlopen(url, timeout=60) as reponse:
                return json.load(reponse)
        except urllib.error.HTTPError as e:
            if e.code != 429 and e.code < 500:
                raise
            erreur = f"HTTP {e.code}"
        except urllib.error.URLError as e:
            erreur = str(e.reason)
        if essai == ESSAIS:
            raise RuntimeError(f"abandon après {ESSAIS} essais ({erreur}) : {url}")
        attente = 2 ** essai
        print(f" essai {essai} raté ({erreur}), nouvel essai dans {attente} s")
        time.sleep(attente)

def main():
    parametres = {"f": "json", "CLIMATE_IDENTIFIER": STATION, "sortby": "LOCAL_DATE",
                  "datetime": f"{DEBUT} 00:00:00/{FIN} 00:00:00", "limit": PAGE}
    url = f"{API}?{urllib.parse.urlencode(parametres)}"
    sortie = PROJET / "plateforme" / "lac" / "bronze" / "meteo_api" / f"quotidien_{STATION}_{DEBUT}_{FIN}.jsonl"
```

```

sortie.parent.mkdir(parents=True, exist_ok=True)

pages = recus = 0
annonces = None
with open(sortie, "w", encoding="utf-8", newline="\n") as fichier:
    while url:
        page = lire_page(url)
        pages += 1
        annonces = page["numberMatched"]
        for entite in page["features"]:
            fichier.write(json.dumps(entite["properties"], ensure_ascii=False) + "\n")
        recus += page["numberReturned"]
        print(f"  page {pages} : {page['numberReturned']} jours")
        # La page suivante, si l'API en annonce une.
        url = next((lien["href"] for lien in page["links"] if lien["rel"] == "next"), None)

print(f"{recus} jours reçus sur {annonces} annoncés, en {pages} pages : "
      f"{sortie.relative_to(PROJET).as_posix()}")
if recus != annonces:
    sys.exit("Écart entre les jours reçus et les jours annoncés : l'extraction est incomplète.")

if __name__ == "__main__":
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")
    main()

```

Morceau	Ce qu'il fait
parametres	la station de Bagotville A, la période, le tri par date, et des pages de 50 jours pour voir la pagination (l'API en accepte jusqu'à 10 000)
lire_page	une requête, et ses reprises : 2, 4, puis 8 secondes après une coupure, un 429 ou une erreur 5xx ; aucune reprise après une autre erreur 4xx
le lien next	la page suivante, telle que l'API la construit ; la boucle s'arrête quand il n'y en a plus
recus != annonces	la preuve qu'on a tout reçu : la somme des numberReturned égale numberMatched

Lancez-le :

```
cd 01_de
python ingestion\extraire_meteo_api.py
cd ..

page 1 : 50 jours
page 2 : 42 jours
92 jours reçus sur 92 annoncés, en 2 pages : plateforme/lac/bronze/meteo_api/quotidien_7060400_2025-10-01_2025-12-31.jsonl
```

Relancer le script réécrit le même fichier : l'extraction est idempotente. Comparez maintenant l'API à l'instantané de la RAW. Rouvrez DuckDB (duckdb) :

```
-- Attendu : 5 colonnes. LOCAL_DATE timestamp, MEAN_TEMPERATURE double, TOTAL_SNOW double,
-- SNOW_ON_GROUND bigint, CLIMATE_IDENTIFIER varchar.
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
USE entrepot;
DESCRIBE SELECT LOCAL_DATE, MEAN_TEMPERATURE, TOTAL_SNOW, SNOW_ON_GROUND, CLIMATE_IDENTIFIER
FROM read_json('plateforme/lac/bronze/meteo_api/*.jsonl');
```

Le JSON de l'API porte des nombres, que DuckDB lit en nombres ; l'instantané de la RAW les garde en texte. Comparez-les d'abord comme du texte :

```
-- Attendu : 92 jours, 5 écarts de température moyenne, 45 écarts de précipitations.
WITH api AS (SELECT * FROM read_json('plateforme/lac/bronze/meteo_api/*.jsonl')),
instantane AS (
    SELECT * FROM raw.meteo_quotidien
    WHERE CLIMATE_IDENTIFIER = '7060400' AND LOCAL_DATE BETWEEN '2025-10-01' AND '2025-12-31')
SELECT COUNT(*) AS jours,
    COUNT(*) FILTER (WHERE CAST(a.MEAN_TEMPERATURE AS VARCHAR) IS DISTINCT FROM i.MEAN_TEMPERATURE) AS temperature,
    COUNT(*) FILTER (WHERE CAST(a.TOTAL_PRECIPITATION AS VARCHAR) IS DISTINCT FROM i.TOTAL_PRECIPITATION) AS precipitations
FROM api a
JOIN instantane i ON i.LOCAL_DATE = CAST(CAST(a.LOCAL_DATE AS DATE) AS VARCHAR);
```

Cinquante écarts, en apparence. Regardez les températures :

```
-- Attendu : 5 lignes. 2025-10-08, 6.0 contre 6 ; 2025-10-09, 3.0 contre 3 ; 2025-10-25, 6.0 contre 6 ;
-- 2025-11-11, -2.0 contre -2 ; 2025-12-07, -19.0 contre -19.
SELECT CAST(CAST(a.LOCAL_DATE AS DATE) AS VARCHAR) AS jour, a.MEAN_TEMPERATURE AS api, i.MEAN_TEMPERATURE AS instantane
FROM read_json('plateforme/lac/bronze/meteo_api/*.jsonl') a
JOIN raw.meteo_quotidien i
    ON i.CLIMATE_IDENTIFIER = '7060400' AND i.LOCAL_DATE = CAST(CAST(a.LOCAL_DATE AS DATE) AS VARCHAR)
WHERE CAST(a.MEAN_TEMPERATURE AS VARCHAR) IS DISTINCT FROM i.MEAN_TEMPERATURE;
```

6.0 et 6 : deux textes différents, un seul nombre. Comparez dans le bon type :

```
-- Attendu : 92 jours, 0 et 0.
WITH api AS (SELECT * FROM read_json('plateforme/lac/bronze/meteo_api/*.jsonl')),
```

```
instantane AS (  
    SELECT * FROM raw.meteo_quotidien  
    WHERE CLIMATE_IDENTIFIER = '7060400' AND LOCAL_DATE BETWEEN '2025-10-01' AND '2025-12-31')  
SELECT COUNT(*) AS jours,  
    COUNT(*) FILTER (WHERE a.MEAN_TEMPERATURE IS DISTINCT FROM CAST(i.MEAN_TEMPERATURE AS DOUBLE)) AS temperature,  
    COUNT(*) FILTER (WHERE a.TOTAL_PRECIPITATION IS DISTINCT FROM CAST(i.TOTAL_PRECIPITATION AS DOUBLE)) AS precipitations  
FROM api a  
JOIN instantane i ON i.LOCAL_DATE = CAST(CAST(a.LOCAL_DATE AS DATE) AS VARCHAR);
```

L’API et l’instantané concordent sur les 92 jours. Quittez DuckDB (.exit).

Étape 5 — Versionner le code, laisser dehors données et secrets

Le code va dans le dépôt ; les données, le .env et l’entrepôt, non. Depuis Projet_Boreal, dans l’Anaconda Prompt :

```
cd 01_de  
git add ingestion sql .env.exemple  
git status  
  
git status liste les deux scripts d’extraction, le contrôle et .env.exemple, prêts à être validés, mais pas .env : la ligne .env du .gitignore du Cours 01 le  
tient dehors. Le docker-compose.yml, lui, n’est pas dans ce dépôt : il vit dans Projet_Boreal/plateforme, avec le reste de la plateforme, que l’équipe qui  
l’exploite tient (le bloc d’ouverture de ce labo). Vérifiez l’exclusion du .env :  
  
git check-ignore -v .env  
git commit -m "Cours 04 : base de l'ERP par ODBC et API météo"  
git status  
cd ..
```

git check-ignore -v .env répond .gitignore:2:.env .env : le fichier est ignoré, par la deuxième ligne du .gitignore. Le dernier git status répond que l’arbre de travail est propre.

Récapitulatif

Vous avez...	Preuve
interrogé la base de l’ERP	boreal_erp sur le port 5433, 25 483 commandes ; un filigrane qui passe par l’index, et un autre qui lit toute la table
lu la base par ODBC et prouvé les extraits	9 tables en Parquet ; controle_erp_base.sql : 0 écart dans les deux sens

Vous avez...	Preuve
éprouvé le filigrane	427 commandes, les mêmes que l'extrait de décembre ; 390 ratées par AEDAT seul
tiré la vraie API	92 jours en 2 pages ; 50 faux écarts en texte, 0 en nombres
versionné	le code, le contrôle et .env.exemple dans 01_de ; .env ignoré

L'état du pipeline de Boréal à la fin du Cours 04 :

Couche	Ce qu'elle contient	Volume	Contrôle
bronze	l'historien (Cours 02)	92 fichiers, 275 436 lignes	CSV contre Parquet
bronze	la télémétrie (Cours 02)	843 fichiers, 158 146 messages	JSON contre Parquet
bronze	la base de l'ERP lue par ODBC	9 fichiers	0 écart avec la RAW reconstruite
bronze	la météo de l'API	92 jours	92 reçus sur 92 annoncés
bronze	le corpus (Cours 03)	35 fichiers	second dépôt : 0 fichier
entrepôt, raw	les 22 livraisons	513 857 lignes	second passage : 0 fichier
entrepôt, pipeline	le journal des chargements	1 572 fichiers	un fichier, une ligne

Prochaine étape (Cours 05). Le chargeur tourne, mais c'est encore un script : le chemin du projet est calculé en dur, les messages vont à l'écran, une coupure de l'API se réessaie dans un coin du code, et rien ne prouve qu'une modification ne casse pas le journal. Le Cours 05 en fait un pipeline : une configuration, des journaux exploitables, des reprises communes, des tests pytest, et une image Docker qui le lance d'une seule commande.

Dépannage

Symptôme	Cause probable	Correctif
Catalog Error: SET schema: No catalog + schema named "entrepot" found.	DuckDB a été rouvert sans l'ATTACH	lancer ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot;; puis USE entrepot;

Symptôme	Cause probable	Correctif
Catalog Error: Table with name "raw.erp_vbak" does not exist because schema "raw" does not exist.	même cause, ou le chargeur n'a pas encore tourné	l'ATTACH, puis l'étape 2 si besoin
L'entrepôt est occupé : fermez la ligne de commande DuckDB, puis relancez., suivi de IO Error: Cannot open file	la ligne de commande DuckDB, ou un autre programme, tient boreal.duckdb ouvert	.exit dans DuckDB, puis relancer le script
IO Error: Failed to open file "01_de/sql/controle_erp_base.sql"	le terminal n'est pas dans Projet_Boreal, ou le fichier s'appelle controle_erp_base.sql.txt	se placer dans Projet_Boreal ; dir 01_de\sql montre le vrai nom
SELECT: command not found, ou -bash: syntax error near unexpected token	du SQL collé dans le terminal d'Ubuntu au lieu de psql	ouvrir psql (docker compose exec erp psql -U postgres -d erp_boreal), puis recoller ; supprimer le fichier = que le >= a créé dans 01_de, avec rm ./=
no configuration file provided: not found	docker compose lancé ailleurs que dans plateforme	dans Ubuntu, cd vers .../Projet_Boreal/plateforme, puis relancer
The "ERP_MOT_DE_PASSE" variable is not set. Defaulting to a blank string.	plateforme/.env manque, ou sa ligne est mal écrite	le refaire comme au bloc d'ouverture ; puis docker compose down -v, docker compose up -d erp et charger_erp.sql, pour réinitialiser la base avec le mot de passe
password authentication failed for user "postgres", à l'extraction ODBC	la base a été initialisée avec un autre mot de passe : POSTGRES_PASSWORD n'agit qu'au premier démarrage du volume	docker compose exec erp psql -U postgres -d erp_boreal -c "ALTER USER postgres PASSWORD 'votre-valeur';" ; ou docker compose down -v, docker compose up -d erp, puis recharger charger_erp.sql
Bind for 0.0.0.0:5433 failed: port is already allocated	un autre conteneur publie déjà le port 5433, par exemple dans Docker Desktop	arrêter ce conteneur, ou quitter Docker Desktop (guide d'installation, étape 5, le piège des deux Docker)
psql répond que la base démarre (the database system is starting up)	PostgreSQL initialise encore sa base	attendre quelques secondes, puis relancer la commande
ModuleNotFoundError: No module named 'pyodbc'	pyodbc n'est pas installé dans formations	conda activate formations, puis pip install pyodbc

Symptôme	Cause probable	Correctif
Pilote psycopg2 introuvable. Pilotes ODBC installés : [...]	psycopg2 n'est pas installé, ou c'est la version 32 bits	reprendre l'étape 7 du guide d'installation du parcours, puis relancer le script
l'extraction ODBC échoue sur une connexion refusée	le conteneur est arrêté : Ubuntu s'est arrêtée à la fermeture de son dernier terminal	rouvrir Ubuntu dans plateforme, docker compose ps, au besoin docker compose up -d erp
extraire_meteo_api.py affiche essai 1 raté, puis abandon après 4 essais	pas d'accès à Internet, ou l'API d'ECCC est indisponible	vérifier la connexion, puis relancer plus tard : le script réécrit le même fichier
controle_erp_base.sql rend des écarts	la RAW ou le bronze erp_base est incomplet : un script s'est arrêté en route, ou les sources ont été régénérées depuis	relancer charger_raw.py, puis extraire_erp_odbc.py, puis le contrôle
git status montre .env parmi les fichiers à valider	le .gitignore du Cours 01 manque, ou n'a pas la ligne .env	corriger le .gitignore (Labo 01, étape 1) avant tout commit

Annexe — Tout le code, d'un seul tenant

Pour rejouer le labo sans relire les explications. Les commandes de terminal se tapent dans l'Anaconda Prompt, depuis Projet_Boreal, sauf celles de la partie 1, dans Ubuntu ; les requêtes, dans la ligne de commande DuckDB ou dans psycopg2, comme indiqué.

1 — La base de l'ERP, déjà montée au bloc d'ouverture (*depuis Projet_Boreal, cd plateforme, wsl, et dans Ubuntu*)

```
docker compose ps erp
docker compose exec erp psycopg2 -U postgres -d erp_boreal
```

Puis, à l'invite erp_boreal=# :

```
SELECT COUNT(*) AS decembre FROM sap.vbak WHERE erdat >= '20251201' OR aedat >= '20251201';
EXPLAIN SELECT * FROM sap.vbak WHERE aedat >= '20251201';
EXPLAIN SELECT * FROM sap.vbak WHERE erdat >= '20251201' OR aedat >= '20251201';
SELECT h.ubdate, h.tcode, p.fname, COUNT(*) AS changements FROM sap.cdhdr h JOIN sap.cdpos p USING (mandt, objectclas, objectid,
changenr) GROUP BY h.ubdate, h.tcode, p.fname ORDER BY h.ubdate;
\q
```

2 — L'extraction par ODBC et son contrôle (*créer 01_de\env, 01_de\env.exemple, 01_de\ingestion\extraire_erp_odbc.py et 01_de\sql\controle_erp_base.sql ; Ubuntu reste ouverte*)

```
python -c "import pyodbc; print(pyodbc.drivers())"
cd 01_de
python ingestion\extraire_erp_odbc.py
cd ..
duckdb -f 01_de/sql/controle_erp_base.sql
```

3 — Le filigrane et l'API (*créer 01_de\ingestion\extraire_meteo_api.py avec le contenu de l'étape 4, puis*)

```
cd 01_de
python ingestion\extraire_meteo_api.py
cd ..
duckdb
```

Puis, à l'invite D :

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
USE entrepot;
SELECT COUNT(*) AS en_tetes FROM raw.erp_vbak WHERE _lot = '2025-12';
.exit
```

4 — Le dépôt (*Anaconda Prompt, depuis Projet_Boreal*)

```
cd 01_de
git add ingestion sql .env.exemple
git check-ignore -v .env
git commit -m "Cours 04 : base de l'ERP par ODBC et API météo"
git status
cd ..
```